

Basic Terminology

Symbols $\{a, b, c, 0, 1, 2, \dots\}$
alphabet (finite, ^{non-empty} set of symbols)
 $\Sigma(a, b) \rightarrow$ finite set of a & b

String $\rightarrow$ finite collection of alphabets, ^{finite} sequence of alphabets
eg- strings from $\Sigma(a, b)$ of length 2

$\{aa, ab, ba, bb\}$

Language $\rightarrow$ collection of strings
Language
finite infinite

Q Given alphabet $\Sigma(a, b)$, find

a) $L_1 =$ strings of length 3

$L_1 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$

b) language L_2 in which all strings start with a & end with a

$L_2 = \{aa, aqa, aaaa, \dots\}$

$\{aba, abba, abbb, \dots\}$

infinite
c) $L_3 =$ string of length 0 (empty string)
 $L_3 = \{\epsilon\}$

Automata

Mathematical model or machine used to find whether string belongs to language or not.

Power of Σ

$\Sigma^0 =$ set of all strings with length 0 $= \epsilon, \lambda$

$\Sigma^1 =$ set of all strings with length 1

$\Sigma^2 =$ set of all strings with length 2

Σ^* (Kleene closure) = infinite language

$\downarrow$
 $(a+b)^*$
all combinations of a & b (total 2^n)
set of all strings of all lengths

Σ^+ (positive closure) $\rightarrow$ all strings except ϵ

$$\Sigma^* = \Sigma^+ + \Sigma^0$$
$$\Sigma^* - \epsilon = \Sigma^+$$

ϵ (identity element or null)

Grammar

A grammar G is defined as quadruple

$G = \{V, T, P, S\}$

V
Variable

T
Terminal

P
Production rules

S
start symbol

grammar is standard way of representing a language

eg - $S \rightarrow aSb \mid \epsilon$

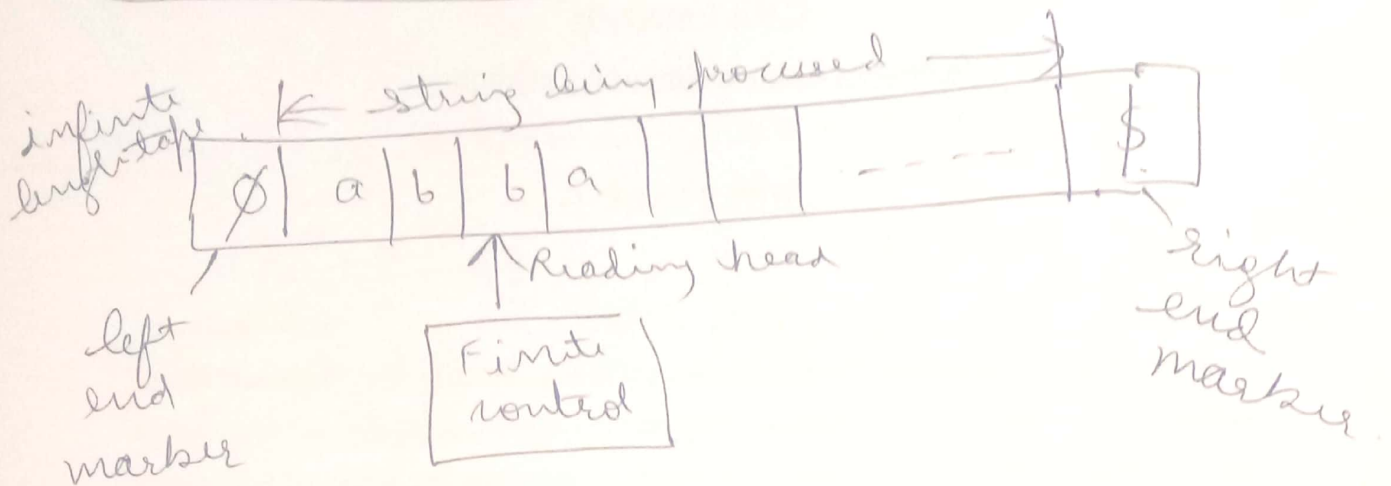
$S \rightarrow aSb$
 $S \rightarrow a\epsilon b$
 $S \rightarrow ab$

$S \rightarrow aSb$
 $S \rightarrow aaSbb$
 $S \rightarrow aaa\epsilon bbb$
 $S \rightarrow aaabbb$
 $S \rightarrow a^2b^2$

$S \rightarrow aSb$
 $S \rightarrow aaSbb$
 $S \rightarrow aaaSbbb$
 $S \rightarrow a^3Sb^3$
 $S \rightarrow a^3b^3$ -----

$L = \{a^n b^n ; n \geq 0\}$

Structural representation of FA



3 components: -

① Input tape

It is divided into squares where each square contains a single symbol from the alphabets.

left side has left hand marker & right side has right end "

② Reading head.

It examines one symbol at a time & can move one square either to the left or to the right.

③ Finite state control

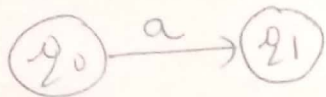
It is responsible for controlling total functioning of FA. It will decide which input symbol is read next & where to move either to the left or to right. The input to finite control will be input symbol from input tape & present state of machine. o/p is movement to next & next state of m/c.

DFA

→ Deterministic FA

$(Q, \Sigma, \delta, q_0, F)$

→ Output is deterministic i.e., ^{to} only one state



Here applying a to state q_0 will change state to q_1 .

→ $Q \rightarrow$ set of finite states

$\Sigma \rightarrow$ set of alphabets

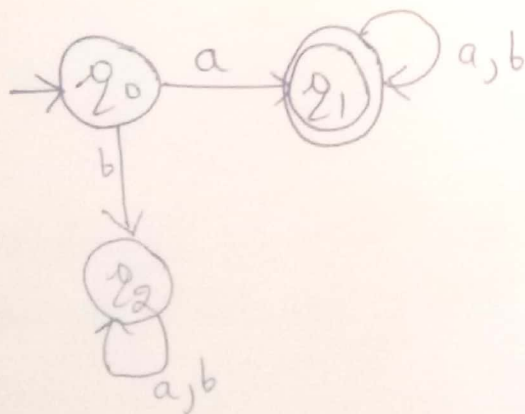
$\delta \rightarrow$ transition function $\delta: Q \times \Sigma \rightarrow Q$

$q_0 \rightarrow$ initial state or starting state

$F \rightarrow$ set of final states

$$F \subseteq Q$$

Q1 Design FA where all strings start with a , $\Sigma = \{a, b\}$
 $L = \{a, aa, aaa, ab, \dots\}$



$$\delta(q_0, a) = q_1$$

$$\delta(q_0, b) = q_2$$

$$\delta(q_1, a) = q_1$$

$$\delta(q_1, b) = q_1$$

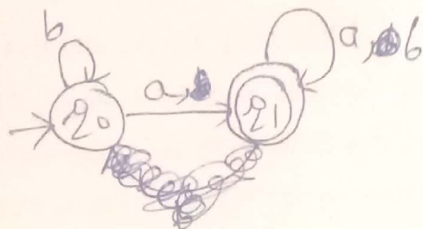
$$\delta(q_2, a) = q_2$$

$$\delta(q_2, b) = q_2$$

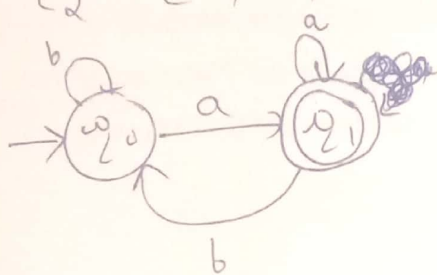
Q2 Construct a DFA which accept a language of all strings with $\Sigma = \{a, b\}$

- containing a
- end with a

Ans $L_1 = \{a, aa, aaa, \dots, ba, baa, aba, ab, abab, \dots\}$

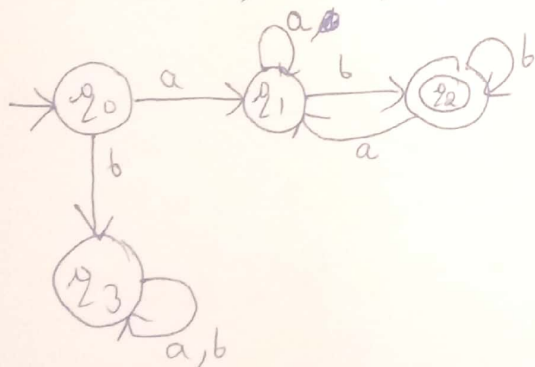


b) $L_2 = \{a, aa, aaaa, \dots, ba, baa, baba, aba, \dots\}$



Q3 Construct a DFA which accept a language of all strings starting with 'a' & ending with 'b'

Ans $L = \{ab, aab, aaab, abab, abatab, \dots\}$



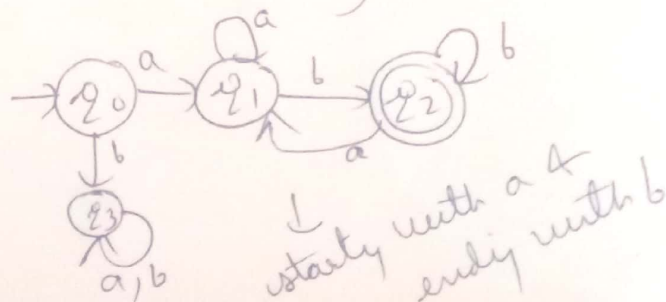
Q4 Construct a DFA which accept a language of all strings ^{not} starting with 'a' or not ending with 'b'

Ans $L = \{ba, \epsilon, baa, \dots\}$

A = not starting with a

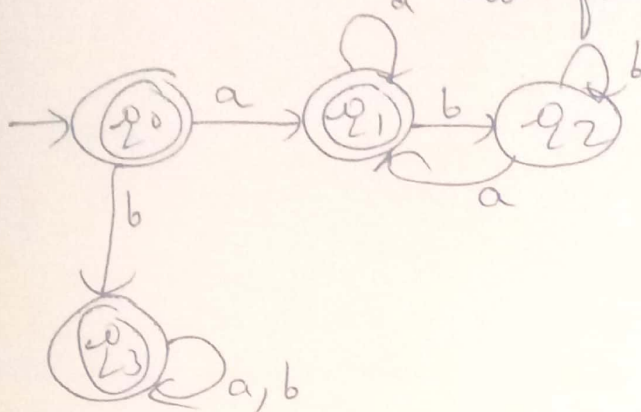
B = not ending with b

$$(A \cup B)^c = A^c \cap B^c$$



Now take complement of it

For this make final $\rightarrow$ non-final
non-final $\rightarrow$ final

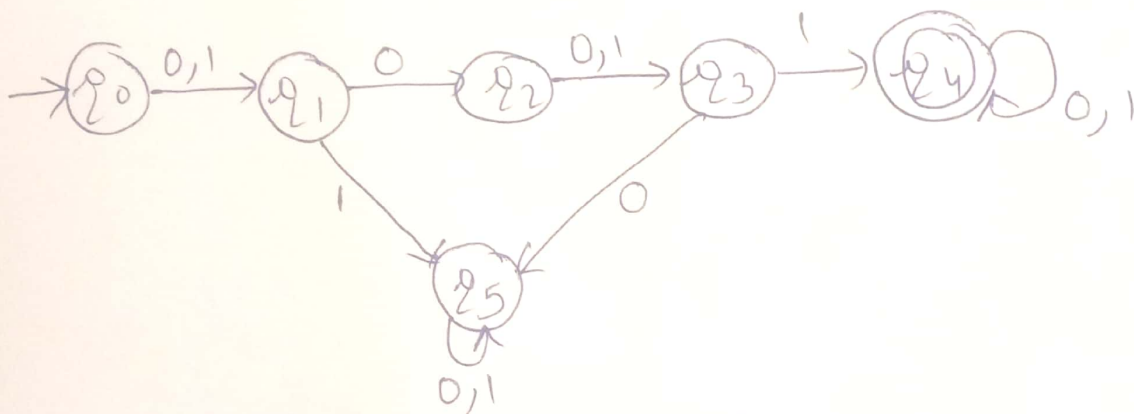


Q5 Design a DFA which accepts all strings over $\{0,1\}$ in which second symbol is '0' & fourth symbol '1'.

Ans

0 1 0 1 1

$$\begin{aligned} \text{minimum length} &= 4 = n \\ \text{minimum states} &= n + 1 \\ &= 4 + 1 \\ &= 5 \end{aligned}$$

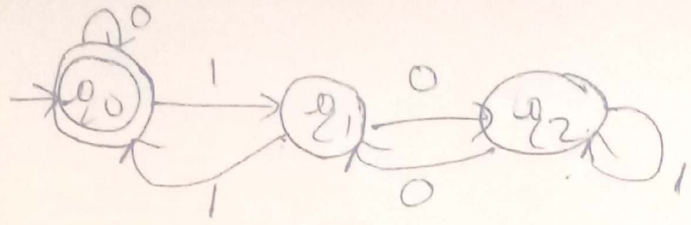


$$\text{Total states} = n + 1 + 1 \rightarrow \text{trap state}$$

Q6 Construct a DFA which accepts a language of all binary strings divisible by 3 over $\Sigma(0,1)$.
When we divide any no by 3 we get 3 remainders 0, 1 & 2.

Ans

Total states q_0, q_1 & q_2



q_0 has all states having remainder 0
 q_1 " " " " " 1
 q_2 " " " " " 2

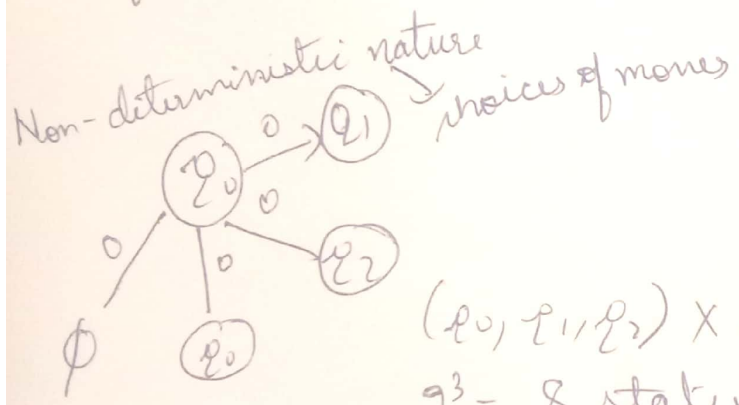
Q Construct DFA accepting strings starting with 'ab' over $\Sigma = \{a, b\}$

NDFA

→ Non-deterministic FA

$$F \subseteq Q$$

$(Q, \Sigma, q_0, F, \delta)$
 ↙ ↘ ↘ ↘
 set of final states input alphabet set of final states transition function
 $Q \times \Sigma \rightarrow F$
 initial state / start state



$$(q_0, q_1, q_2) \times \Sigma (0, 1)$$

$2^3 = 8$ states $q_0, q_1, q_2, q_0q_1, q_0q_2, q_1q_2$

2^3

q_0, q_1, q_2

DFA

NDFA

① Read configuration is not allowed.

② Multiple choices are not possible corresponding to I/P

③ E-move is not allowed

④ Design difficult

Read configuration is allowed

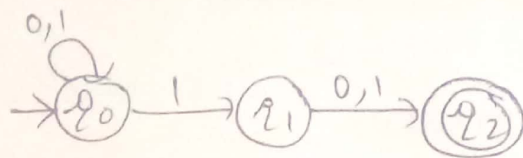
Multiple moves available corresponding to an input

E-move allowed.

Design is easy.

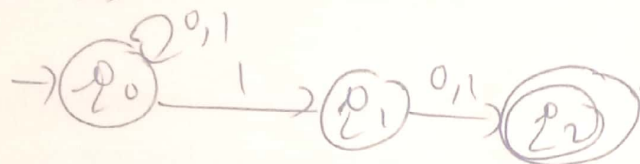
Q Design NFA of all binary strings in which second last bit is 1.

Ans $(0+1)^* 1 (0+1) \rightarrow$ regular expression



Convert NFA to DFA

NFA of all binary strings in which 2nd last bit is 1

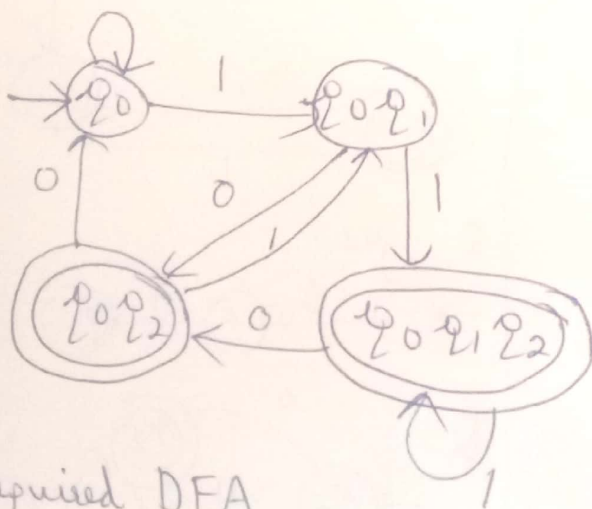


	0	1
$\rightarrow q_0$	q_0	$q_0 q_1$
q_1	q_2	q_2
q_2	—	—

Transition table of NFA

	0	1
$\rightarrow q_0$	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_2$	$q_0 q_1 q_2$
$q_0 q_2$	q_0	$q_0 q_1$
$q_0 q_1 q_2$	$q_0 q_2$	$q_0 q_1 q_2$

Transition table of DFA



Required DFA

If in NFA we have n states, then we have 2^n states in its equivalent DFA

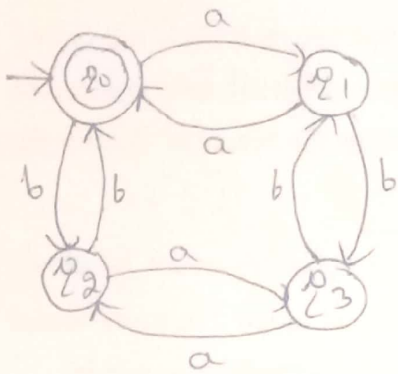
Q Let $L = \{w \mid w \text{ has even no. of } a\text{'s} \text{ \& even no. of } b\text{'s}\}$

(9)

Design a DFA which accepts L .

Ans

$L = \{ \epsilon, aa, bb, aabb, abab, bbaa, baba, abba, baab, \dots \}$



If q_1 is final, it will accept odd no. of a & even no. of b

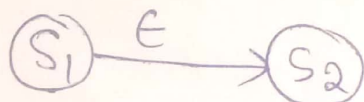
If q_2 is final, it will accept even no. of a & odd no. of b

If q_3 is final, it will accept odd no. of a & odd no. of b

Eliminate null moves from NFA

OR

Convert NFA with ϵ -moves to NFA without ϵ -m



To remove, do steps

- ① Find all edges starting from S_2
- ② Duplicate all edges to S_1 without changing

edge labels.

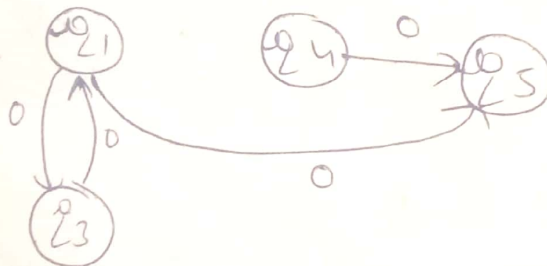
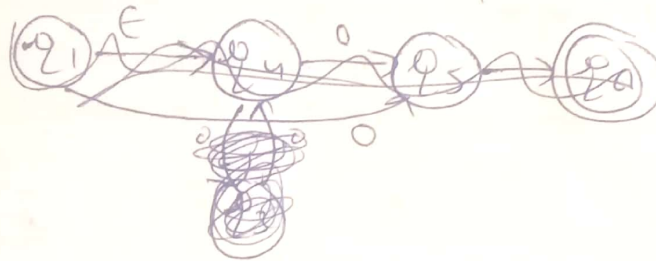
(10)

- ③ If S_1 is initial state, make S_2 as initial
- ④ If S_2 is final state, make S_1 as final state

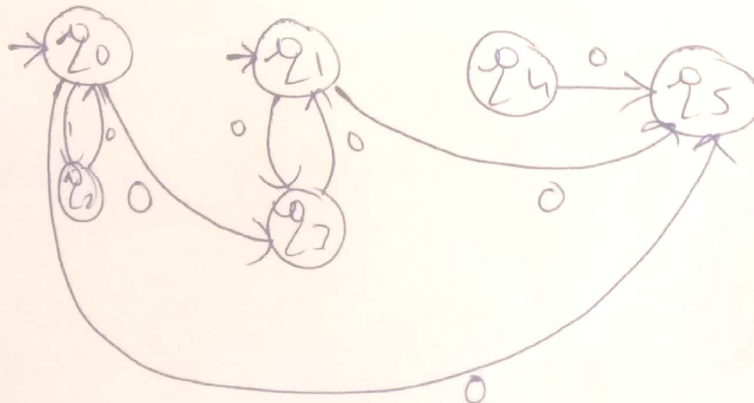
eg -



First remove null move b/w q_1 & q_4



Remove ϵ move b/w q_0 & q_1



⑤ Remove dead state

Here q_4 is dead state since it has no incoming edge.

Limitations of Finite State Automata

11

① Limited memory

② Strings without comparison can be only accepted.

Eg - $a^n b^n$ cannot be accepted as it requires memory to compare no of a & b

$a^m b^n$ can be accepted

But $a^m b^n$ where $m=n / m \neq n / m > n / m < n$ cannot be accepted

③ Linear power strings are accepted

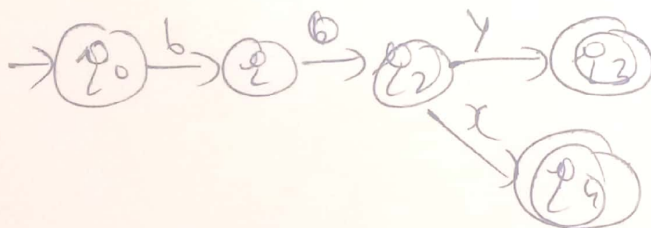
Eg - a^n

a^{2n} , a^i , a^{n^2} cannot be accepted
prime no

Applications of FSA

① Word processor program.

boy, box



② Digital logic design

③ Lexical analyzer

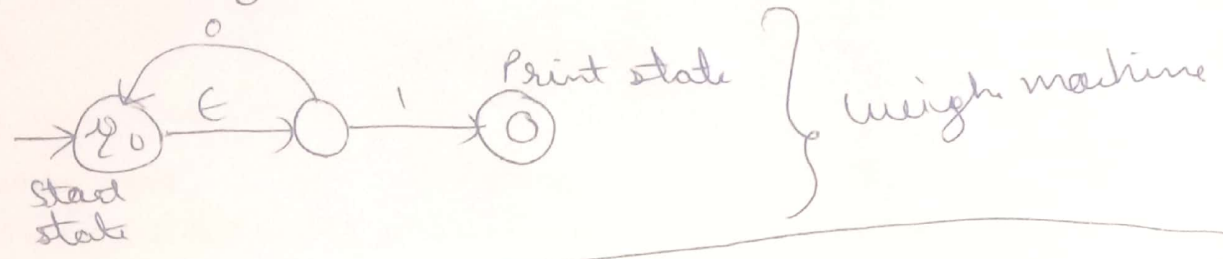
④ switching circuit design

⑤ Text editors

⑥ Software for scanning large bodies of text such as web pages to find occurrence of

words, phrases, patterns

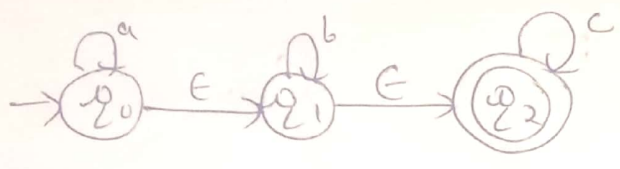
- ⑦ Software with finite states like vending m/c, weigh machine, traffic lights, toll machine etc
- ⑧ game design



NFA with E-moves or E-NFA

$(Q, \Sigma, q_0, F, \delta)$

$\delta: Q \times (\Sigma \cup \epsilon) \rightarrow 2^Q$

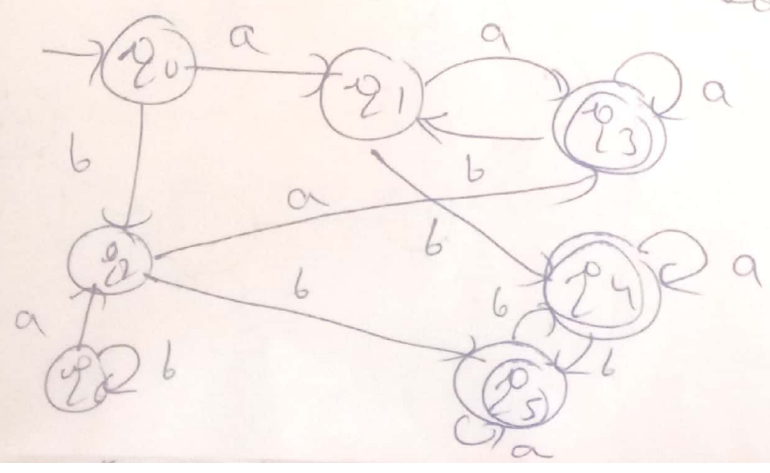


Minimization of DFA

When we convert NFA to DFA, no. of states increases

NFA $\rightarrow$ DFA
 $\downarrow$
n states $\quad 2^n$ states
 $\downarrow$

makes system complex
so need to be reduced



- ① Remove unreachable state from q_0
 q_6 cannot be reached through q_0 .
 So, remove q_6

	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_3	q_4
q_2	q_3	q_5
q_3	q_3	q_1
q_4	q_4	q_5
q_5	q_5	q_4

- ② Remove all states in equivalent classes
- accepting
final
states

non-accepting

$\{q_0, q_1, q_2\}$ non-accepting

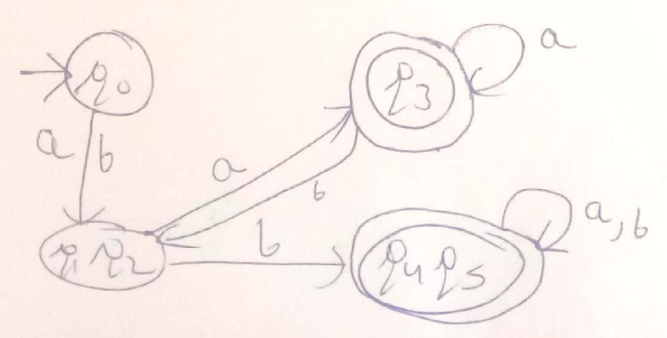
$\{q_3, q_4, q_5\}$ accepting

$$\pi_0 = \{q_0, q_1, q_2\} \{q_3, q_4, q_5\}$$

$$\pi_1 = \{q_0\} \{q_1, q_2\} \{q_3\} \{q_4, q_5\}$$

$$\pi_2 = \{q_0\} \{q_1, q_2\} \{q_3\} \{q_4, q_5\}$$

$$\pi_1 = \pi_2$$



Minimized DFA